Grammar-Constrained Decoding for English to ASL Gloss Translation

Project Report – Natural Language Processing (Prof. Misael Mongiovì)
Giovanni Maria Manduca – LM18 – 1000069285
University of Catania

Abstract

Sign language glosses are essentially written labels for signs. A system that produces glosses for another one that consumes them, such as a signing avatar, has to stay inside a certain vocabulary: an unknown gloss cannot be signed. This report asks whether forcing a translation model to respect a closed gloss vocabulary through grammar-constrained decoding costs translation quality, and where that cost comes from. I compare four English to ASL-gloss systems on the ASLG-PC12 corpus: zero-shot and few-shot prompted Llama-3.2-3B, a fine-tuned BART-base, and a QLoRA-tuned Llama-3.2-3B. Each is decoded with and without an xgrammar constraint, with varying iterations of grammar and vocabularies considered. The constraint always gives valid output, but reduces the quality of the output. Still, this cost shrinks as the model gets better: about 11 chrF points for zero-shot, 7 for few-shot, 7 for BART, and only under 1 for the QLoRA model. On the QLoRA model the whole remaining cost comes from references that the grammar cannot express; on every reference it can express, constrained decoding is at least as accurate as free decoding. Widening the grammar with the copy rule and a compositional `DESC-` rule closes the gap: the constrained QLoRA model matches the unconstrained one on BLEU and exact match, with 100% valid output, greatly outperforming the encoder-decoder method considered even with a much smaller training split.

1 Introduction

1.1 Gloss

A gloss is a written label for a sign. Sign languages have no widely used written form, so linguists write a sign with an upper-case word of the surrounding spoken language, chosen for its approximate meaning: the ASL sign for *house* is written `HOUSE`. The practice comes from the first linguistic descriptions of ASL. Stokoe [1] showed in 1960 that ASL is a full language with its own structure, and proposed a phonetic notation for it; glossing became the everyday alternative to such notations since it can be typed and read without training. The conventions used today in ASL textbooks (upper-case labels, hyphens for multi-word signs, prefixes and suffixes for grammatical markers) come largely by teaching material of the 1980s, and later carried into annotated sign language corpora.

A gloss line is not a translation. It records which signs appear and in what order, while dropping much of what the signer's face, body and space express. The two examples below are taken from the evaluation data used in this report (ASLG-PC12 conventions: `X-` for pronouns, `DESC-` for descriptive words such as adjectives, remaining verbs and nouns reduced to their lemma):

```
here I refer, in particular, to health and
education policies.
DESC-HERE X-I REFER , IN DESC-PARTICULAR ,
TO HEALTH AND EDUCATION POLICY .

they will be equal in every sense.
X-Y WILL BE DESC-EQUAL IN EVERY SENSE .
```

Glosses matter for language technology because they are the natural intermediate step between text and sign. Sign language synthesis systems first translate text into

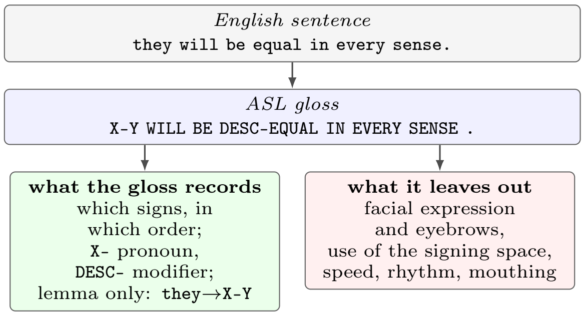

Figure 1: What a gloss is: a written label per sign, in sign order, and nothing of the non-manual channel.

a gloss sequence, then they turn each gloss into motion for an avatar or a generated video. Sign language recognition and translation systems often go the other way, from video to glosses to text [2]. The gloss vocabulary is closed for both directions: the animation model only knows a fixed list of signs (with possible fallbacks for stuff like numbers), and a gloss outside that list cannot be rendered. This is what makes output validity a hard requirement for such tasks.

1.2 Machine translation

Machine translation is defined as the task of mapping a sentence in a source language to one in a target language. Neural machine translation models the target sentence one token at a time with an encoder-decoder network (BART) or, as is the SOTA for pretty much every nlp generation task in recent years, with a decoder-only architecture. Both these are based on the transformer architecture. I compared two families of models:

- **Encoder-decoder models trained for the task.** BART [4] is a pretrained Transformer encoder-decoder

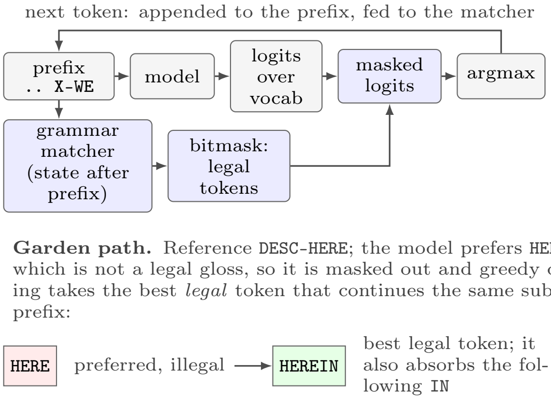

Figure 2: Grammar-constrained decoding with a token bitmask, and the garden-path failure.

that is fine-tuned end to end on source and target pairs. It is small (140M parameters) and fast.

- **Decoder-only large language models.** Llama-3.2-3B-Instruct [5] is a general chat model. In particular I use QLoRA [6] for the finetune purposes: the base model is stored in 4-bit precision and only small low-rank adapters are trained.

Translation quality is measured with BLEU [7], chrF [8] and ROUGE-L [9], plus exact match.

1.3 Constrained decoding

A language model produces text by picking one token at a time from its vocabulary. Constrained decoding changes the choice at each step: tokens that would take the output outside a target language, described by a regular expression or a context-free grammar, get their probability set to zero before the next token is picked [11, 12]. The model still decides among the legal continuations; the constraint only removes the illegal ones. Thus, the result is guaranteed to belong to the target language by construction.

One of the problem I encountered is speed: the grammar is written over characters, while the model emits sub-word tokens, so the engine must know, for every grammar state, which of the tens of thousands of tokens are allowed. xgrammar [13] precomputes most of this per tokenizer and applies the mask on the GPU, which is why it can be used for a vocabulary of fifteen thousand gloss words effectively without making decoding prohibitively costly.

Constrained decoding has a known failure mode that will matter in the results. With greedy decoding, when the model's preferred token is illegal, the mask forces the best *legal* sub-word prefix, which can be the beginning of a completely different word: an intended `HERE` becomes `HEREIN`, an intended `IS` becomes `ISSOCIALISM`. I call this a *garden path*. It cannot happen when the intended word is legal, so it should become rarer as the model gets better (and the grammar does too).

	Pairs
Raw (Kaggle CSV)	87,710
After exact deduplication	81,090
Train (80%)	64,872
Validation (10%)	8,109
Test (10%)	8,109
Evaluation subset (first 1,000 of test)	1,000
Gloss vocabulary (train only)	14,741 types

Table 1: Corpus statistics after cleaning and splitting.

2 Dataset

2.1 ASLG-PC12 data card

Name and source
ASLG-PC12, English–ASL Gloss Parallel Corpus 2012 [14]. The English side comes from the Europarl proceedings [15]; the gloss side was generated by a rule-based system that rewrites the part-of-speech tagged English sentence into ASL gloss order and conventions.

Distribution used
The Kaggle copy of the corpus [16], a single CSV of 87,710 sentence pairs with two columns (`text`, `gloss`) and no official train/test split.

Language and conventions
English source text, fully lower-cased. Gloss side in upper case with corpus-specific markers: `X-` for pronouns and pointing signs (21 forms, e.g. `X-WE`, `X-POSS`), `DESC-` for adjectives and adverbs (3,101 forms in the training vocabulary), lemmatised verbs and nouns, articles and most prepositions are removed, while punctuation is kept as tokens.

Size and length
81,090 unique pairs after removing exact duplicates. Sentences are short: 13.7 source words and 12.3 gloss tokens on average (medians 14 and 12, maxima 59 and 48).

Vocabulary
14,741 gloss token types in the training split (Table 1).

Known issues
Encoding artefacts (byte-order marks glued to tokens, mojibake such as `Ã€VIS`), about 300 glued noise tokens (`16COULD`), and 6,620 verbatim duplicate pairs.

Intended use here
English to gloss translation, with the gloss vocabulary treated as a closed set.

2.2 Note on contamination

Two kinds of contamination could inflate the results.

Within the corpus. The Kaggle file contains 6,620 exact duplicate pairs (7.55%). Splitting before removing them would put identical pairs on both sides of the train/test boundary. I therefore deduplicate first and split afterwards. On the 1,000-sentence evaluation subset, no source or target sentence (english/gloss) appears anywhere in the training split. What remains is template-level overlap: sentences that differ by one or two content words would cross the split, because the corpus is built from parliamentary speech with possibly recurrent phrasing. This is not quantified.

From pre-training. ASLG-PC12 has been public since 2012, so the Llama-3.2 pre-training data may contain it, test sentences included. If the model had memorized it, prompting alone should surface the glosses. It does not: zero-shot Llama reaches 7.6 BLEU and gets 4 sentences in 1,000 exactly right (Section 4.3), so the fine-tuning gains cannot be explained by memorization of the test set alone. They are probably a mix of genuine learning and the resurfacing of knowledge the model may have picked up in pre-training.

3 Methodology

3.1 Data cleaning and splits

Cleaning is deliberately light. I strip byte-order marks, normalize whitespace, and upper-case every gloss. Mojibake and glued tokens are kept, because they are part of the references and fixing them would need a manual pass outside which is outside the scope of this project. Exact duplicate pairs are removed, and the 81,090 unique pairs are split 80/10/10 with a fixed seed (Table 1). All systems are evaluated on the same subset of 1,000 test sentences; the full test split was not decoded for reasons of compute. The gloss vocabulary that defines a legal token is built from the training split only. This choice, while being the most scientifically honest, has the drawback of removing some reference tokens of the test set from the training vocabulary (e.g. uncommon nouns or topics which might appear only in one or two sentences which landed all in the evaluation set), so a constrained decoder can never reproduce them (Section 3.3.2).

3.2 Architectures

Four systems, in increasing order of task-specific training:

1. **Zero-shot Llama-3.2-3B-Instruct.** The instruction-tuned 3B model, loaded in 4-bit NF4 with double quantisation, no training. The system prompt describes the task and the corpus conventions (Section 4.2).
2. **Few-shot Llama-3.2-3B-Instruct.** The same model and prompt, with five fixed worked examples inserted as user/assistant turns before the query. The examples are the same for every test sentence, so the two prompted systems differ only in the presence of the examples.
3. **Fine-tuned BART-base.** Full fine-tuning of the 140M-parameter encoder-decoder on all 64,872 training pairs, plain text in and gloss out.
4. **QLoRA Llama-3.2-3B-Instruct.** The 4-bit model with LoRA adapters (r=16, α=32, dropout 0.05) on all attention and MLP projections, trained for one epoch on the first 20,000 of the 64,872 training pairs (the full split was not used due to hardware limitations) in the same chat format as the zero-shot prompt.

Each system is decoded under three conditions: *unc* (unconstrained greedy decoding), *con* (greedy decoding under the gloss grammar), and *copy* (greedy constrained decoding, with the grammar widened per sentence with the copy rule described in Section 3.3.1). Nothing is retrained between conditions: the grammar acts only at decoding time.

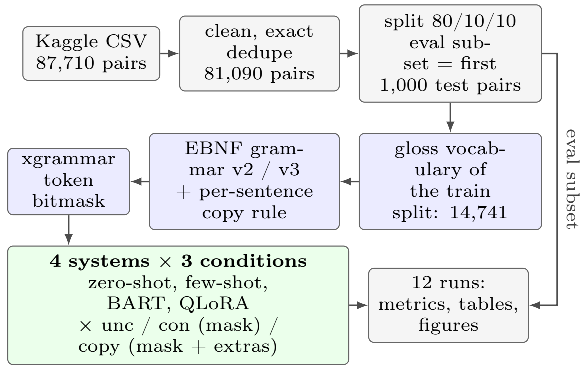

Figure 3: Experimental pipeline. The grammar is built from the training vocabulary only and acts at decoding time.

3.3 Grammars

A legal output is a sequence of legal gloss tokens separated by single spaces. All grammar versions share the skeleton

```
root ::= " "? gloss (" " gloss)*
```

The optional leading space is a tokeniser detail: both BART's BPE and Llama's tokeniser mark the start of a word with a space, so without it the first gloss would be forced through word-internal sub-word pieces (in turn, surfacing even more of the garden path cited before). The versions differ in what counts as a `gloss`:

v1 `gloss ::= word`, with all 14,741 training-vocabulary tokens listed as string literals. Quickly discarded due to the fact that considering the numbers as literals is both inefficient and limiting as far as the quantity of representable numbers (considering that, in the training set, about 600 distinct literals would be considered).

v2 Numbers are an open class, so they become a pattern instead literals: `gloss ::= number | word`, with `number ::= [0-9]+ (("." | ",") [0-9]+)*` and `word` the 14,139 remaining tokens.

v3 Adds a compositional rule for the productive `DESC-` prefix on top of v2: `gloss ::= number | word | "DESC-" word`. A reference such as `DESC-CONDUCIVE`, whose stem is in the vocabulary but whose prefixed form is not, becomes expressible. v3 is the grammar of the main results; v2 is kept as an ablation on the fine-tuned systems.

The grammar compiles in about two seconds per tokeniser. The validity metrics use the same definition as the grammar of the run, so "valid" always means "accepted by the grammar that was used".

3.3.1 The copy rule

Roughly half of the reference tokens that the vocabulary lacks are present, lower-cased, in the source sentence:

		Eval subset		Validation	
Grammar	Cond.	Seq. cov.	Tok. OOV	Seq. cov.	Tok. OOV
v1	plain	91.8%	0.84%	90.8%	0.89%
v2	plain	92.2%	0.78%	91.3%	0.85%
v3	plain	92.2%	0.78%	91.3%	0.85%
v2	+ copy	95.9%	0.38%	95.3%	0.40%
v3	+ copy	**97.7%**	**0.23%**	**97.2%**	**0.25%**

Table 2: Coverage of the reference glosses by each grammar: share of references fully expressible (the exact-match ceiling of a constrained decoder) and share of reference tokens the grammar cannot produce. 1,000 sentences per column group.

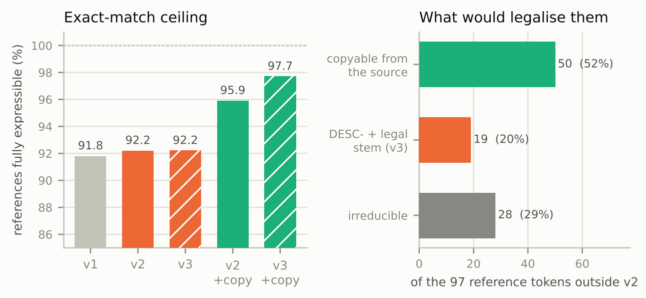

Figure 4: Coverage ceiling by grammar version on the evaluation subset (left; hatched = v3), and the 97 reference tokens grammar v2 cannot produce, sorted by what would legalise them (right).

proper nouns such as `gaza`, and other words the rule-based generator passed through unchanged. The copy rule builds, for each input sentence, a grammar that additionally admits selected source words (upper-cased) as legal glosses. The selection uses simple heuristics to exclude: stopwords, words already in the vocabulary, inflections of a word in the vocabulary (e.g. `suffering` is not copied since `SUFFER` exists), and `DESC-` or `X-` forms of words in the vocabulary. On the evaluation subset 321 sentences have at least one candidate, giving 216 distinct grammars; these are compiled once and cached (about seven minutes per tokeniser). The rule can also over-generate: a candidate the reference does not use is one more chance for a garden path.

In real ASL an unknown proper noun would be fingerspelled (a `FS-` gloss). That would be the linguistically right rule, but this corpus has no fingerspelled forms.

3.3.2 Grammar coverage

A constrained decoder can produce a reference exactly only if the grammar accepts it, so the share of references the grammar accepts is a hard upper bound on exact match. Table 2 reports it for every version, on the 1,000-sentence evaluation subset and, as a check, on the first 1,000 validation sentences.

Under v2, 922 of the 1,000 evaluation references are expressible; the other 78 contain at least one of 97 tokens missing from the training vocabulary. Sorting those 97 tokens by what would make them legal gives the design space: 51.5% are copyable under the copy rule from the source sentence, 19.6% are a `DESC-` form of a stem that is in the vocabulary or copyable, and 28.9% are irreducible (mojibake, glued noise such as `DESC-20IF`). This explains the table. The number rule of v2 recovers four references over v1. v3 alone does not move the ceiling, because every stem it could compose lives in the source sentence rather than in the vocabulary; combined with the copy rule it raises the ceiling from 95.9% to 97.7%.

3.3.3 Implementation

For the causal Llama models, xgrammar's stock Hugging Face logits processor is used unchanged. BART is an encoder-decoder whose decoder starts with forced special tokens. These would be rejected by the xgrammar; the stock processor would feed them to the grammar matcher and reject them. I added a small seq2seq logits processor that skips the forced prefix before advancing the matcher. Both paths were checked with a smoke test on tiny models, decoding under the grammar and verifying the output with the matcher.

4 Experiments

4.1 Settings

All experiments ran on one RTX 3080 (10 GB) under Windows 10, with `torch 2.11`, `transformers 5.15`, `peft 0.20`, `bitsandbytes 0.50` and `xgrammar 0.2.3`. Decoding is greedy everywhere, with a budget of 128 new tokens (the longest reference is 72 Llama tokens and 89 BART tokens).

BART-base: 3 epochs on the 64,872 training pairs, effective batch 32 (16 × 2 gradient accumulation), peak learning rate 3×10^{-5} with linear decay and 300 warm-up steps, fp16 and a maximum length 128 tokens for both sides (the longest training gloss is 117 tokens).

QLoRA Llama: one epoch on the first 20,000 training pairs, effective batch 16 (2 × 8), peak learning rate 2×10^{-4} with a cosine schedule and 40 warm-up steps, paged 8-bit AdamW, gradient checkpointing, maximum sequence length 384 tokens. The prompt alone is about 155 tokens because the system instruction sits on top of every example, and the longest training example is 301 tokens, so no training example is truncated. Validation loss fell to 0.017; training took 87 minutes.

Decoding batches: the prompted systems were decoded at batch size 1 while the GPU was shared with other work; BART at batch 16 and QLoRA at batch 8. Throughput figures are therefore comparable within a system but not across the prompted/fine-tuned boundary.

Metrics: corpus BLEU and chrF (sacreBLEU [10]), sentence-averaged ROUGE-L F_1, exact match, token validity, and sequence validity (share of non-empty outputs whose every token is accepted). Every configuration was run once; differences between conditions are assessed with a paired bootstrap over the 1,000 sentences (1,000 resamples, seed 0).

4.2 System prompt and demonstrations

The two prompted systems and the QLoRA model share the same system prompt, shown below. It was drafted with the help of an LLM. The QLoRA model is trained and evaluated with the zero-shot form of the prompt (no demonstrations).

```
You are a translator from English into
American Sign Language (ASL) gloss
notation, following the conventions of the
ASLG-PC12 corpus.
Rules:
- Output ONLY the gloss translation:
uppercase tokens separated by single
spaces.
- Pronouns and pointing signs use the X-
prefix (e.g.  X-I, X-YOU, X-IT).
- Adverbs/adjectives may use the DESC-
prefix (e.g.  DESC-NEW).
- Do not output explanations or lowercase
words.
```

The user turn is `Translate to ASL gloss: <sentence>`. The few-shot system inserts five training pairs (20 to 90 characters long, drawn once with a fixed seed) as alternating user and assistant turns between the system prompt and the query; two of them are:

```
we are now seeking to rally all our key
players around the strategy .
X-WE BE DESC-NOW SEEK TO RALLY ALL X-WE KEY
PLAYER AROUND STRATEGY .

what do we want to commemorate , and how ?
WHAT DO X-WE WANT TO COMMEMORATE , AND HOW
?
```

4.3 Results

Table 3 is the main result: four systems, three decoding conditions, grammar v3.

Fine-tuning dominates prompting. Zero-shot Llama reaches 7.6 BLEU; five demonstrations lift it to 31; fine-tuned BART reaches 76.7 BLEU and 44.5% exact match; the QLoRA model, the same 3B network that scored 7.6 zero-shot, reaches 98.1 BLEU and 92.0% exact match after one epoch on 20,000 pairs. The corpus is regular enough that a model trained on it learns the generating rules almost completely. Model size still matters among fine-tuned models: BART, trained on three times more data for three epochs, stays far behind and spits out rare words character by character (`JIHADI` → `JIJIRAQIST`), whereas the 3B model, which has seen those words in pre-training, does not.

Demonstrations teach the format while instructions are largely ignored. Both prompted systems receive the same explicit rules (upper case only, `X-` pronouns, `DESC-` modifiers, no explanations). Zero-shot largely ignores them: only 49% of its tokens are legal glosses and it often continues with prose or falls into repetition loops. Five examples are enough to raise token validity to 76%., but do not fix the one or two near-miss tokens per sentence (`REFERR`, `ALIEVATE`, `SUFFERING` for `SUFFER`), so whole-sentence validity stays low (under 6%) for both prompted conditions. For a consumer that needs whole legal sentences, prompting alone is therefore not a viable solution.

The constraint delivers validity on every system. All constrained runs have 100% token validity, and 100% sequence validity on the fine-tuned systems. The fine-tuned models need it: I had expected a model trained on the corpus to stay inside the vocabulary by itself, but this is false (especially for the smaller model): unconstrained BART emits at least one illegal token in about half of its outputs, and the QLoRA model is invalid in 10% of its outputs.

The quality cost shrinks with model quality. Reading down Table 3, the chrF cost of the plain constraint goes from 11 points (zero-shot) to 7 (few-shot) to 7 (BART) to 0.6 (QLoRA). For the prompted models the cost is not in BLEU at all (few-shot even gains 0.9 BLEU; the bootstrap interval of the difference includes zero, Table 6) but in chrF and ROUGE-L. The mechanism is the garden path I talked about before: when the model intends an illegal near-miss such as `NOTORIOUS` (reference `DESC-NOTORIOUS`), the mask admits only legal prefixes and greedy decoding commits to `NOTO RIOT USUALOVER` (a rather catastrophic failure). On the sentences whose free output is fully legal, the constrained output is identical (898 of 898 for QLoRA, 479 of 486 for BART, 55 of 57 for few-shot), on the others the two outputs first diverge at the first illegal token in 97 to 100% of cases, and the token forced there is a different legal word sharing the prefix of the intended one in 554 (zero-shot), 664 (few-shot), 486 (BART) and 91 (QLoRA) of the 1,000 sentences, against a correction to the reference word in 278, 139, 19 and 9. The cost is then the garden path plus the derailment that follows it, since the model keeps conditioning on the forced word (Section 4.4). On the prompted models the constraint never turned a correct output into a wrong one; it only made wrong outputs differently wrong, and occasionally right (`VOTE 17 SEPT 2009` → `VOTE 17 SEPTEMBER 2009`). For BART and QLoRA the constraint costs BLEU as well, and the paired bootstrap says the cost is real (Table 6).

Remaining costs. Table 4 splits the references by whether grammar v2 can express them (922 reachable, 78 unreachable) and reports every constrained condition of the two fine-tuned systems under both grammar versions. For the QLoRA model the constrained decoder is at least as accurate as the free one on the reachable references; its whole exact-match deficit comes from the 78 unreachable references, where it scores zero by construction while the unconstrained model scores by emitting out-of-vocabulary tokens, the same behaviour that makes its outputs invalid. The same direction holds for BART. A hybrid decoder (generate unconstrained output, check validity and regenerate constrained if invalid) gains nothing over a plain constrained decoding run on the QLoRA model and a very little amount on BART (Table 5). The remaining cost of the constraint on a good model is then not a decoding problem but a coverage problem. The correct way to go about this is to

System	Cond.	BLEU	chrF	ROUGE-L	Exact	Tok. val.	Seq. val.	tok/s
	unc	7.56	**47.65**	**48.79**	0.4	49.0	3.3	23
Zero-shot Llama 3B	con	7.73	36.35	37.19	0.5	99.7	95.0	20
	copy	**8.32**	37.29	37.88	**0.6**	99.7	95.9	13
	unc	31.03	**73.67**	**73.27**	1.2	75.8	5.7	14
Few-shot Llama 3B (k=5)	con	31.96	66.77	65.96	**1.8**	100.0	99.7	18
	copy	**32.26**	67.61	66.34	1.7	100.0	99.7	19
	unc	**76.68**	**93.26**	**95.57**	**44.5**	90.2	48.6	1478
Fine-tuned BART-base	con	73.24	85.90	89.50	42.8	100.0	100.0	110
	copy	74.05	86.29	89.93	**44.5**	100.0	100.0	95
	unc	**98.08**	**99.28**	99.15	92.0	99.0	89.8	61
QLoRA Llama 3B	con	96.68	98.64	98.43	87.9	100.0	100.0	47
	copy	98.04	99.27	**99.18**	**92.2**	100.0	100.0	44

Table 3: Main grid on the 1,000-sentence evaluation subset. *con* and *copy* use grammar v3. Validity columns are percentages and use the v3 definition for every row. Throughput of the prompted systems was measured at batch size 1 on a shared GPU, that of the fine-tuned systems at batch 16 (BART) and 8 (QLoRA). The small residual sequence invalidity of the constrained prompted runs comes from outputs that loop and hit the 128-token budget mid-token, not from grammar leakage.

widen the grammar and improve the data curation.

Widening the grammar closes the gap. The copy rule is the best constrained condition on all four systems: best BLEU of the three conditions for both prompted models, and a clear gain over the plain constraint on BART and QLoRA, while maintaining the 100% validity. Adding the `DESC-` rule (v3) on top of copying recovers further unreachable references on the QLoRA model reaching 98.04 BLEU and 92.2% exact match, against 98.1 and 92.0% unconstrained, and the bootstrap interval of the difference includes zero (Table 6). The constrained model is statistically indistinguishable from the free one on quality, but still guarantees 100% sequence validity, whereas the unconstrained model fails on 10% of outputs. As Table 2 predicted, v3 without copying changes almost nothing, because the stems it needs come from primarily from the source. On BART the picture is more mixed: v3 raises chrF and ROUGE-L by about one point and its copy condition matches the unconstrained model's exact match rate, but BLEU drops by one point. It seems like the looser grammar opens new garden paths for a weaker model which is often unsure of the correct token.

Cost in speed. The overhead of the mask is roughly constant per decoding step, so its relative cost depends largely on how expensive the model's forward pass actually is. On the 3B model, this overhead is modest: within a few percent for the prompted systems at batch 1, and about 25% for QLoRA at batch 8; the per-sentence copy grammars add some more because the grammar changes from sentence to sentence. On BART-base, whose forward pass is two orders of magnitude cheaper, the mask cost instead dominates and throughput drops about twelve-fold (1,478 to 123 tokens per second at batch 16).

4.4 Qualitative examples

Selected outputs (*unc* vs. *con*, grammar v3) for the mechanisms discussed above. All were found by the garden-path cell of the companion notebook: the free output is within one token of the reference and contains exactly one illegal token.

Garden path on a function word (few-shot). `IS` is illegal (the corpus lemmatises to `BE`); the mask keeps the prefix and greedy decoding completes it into another word.

src `that is the situation .`

ref `THAT BE SITUATION .`

unc `THAT IS SITUATION .`

con `THAT ISSOCIALISM .`

Garden path that swallows the next tokens (few-shot). `HERE` is illegal without its `DESC-` prefix; `HEREIN` absorbs the `, IN` that followed.

src `here , in cohesion policy , we must strive for this harmony .`

ref `DESC-HERE , IN COHESION POLICY , X-WE MUST STRIVE FOR THIS HARMONY .`

unc `HERE , IN COHESION POLICY , X-WE MUST STRIVE FOR THIS HARMONY .`

con `HEREIN COHESION POLICY X-WE MUST STRIVE FOR THIS HARMONY .`

Garden path followed by derailment (few-shot). One illegal token, and the rest of the sentence collapses (chrF 92 → 34).

src `we also need to look forwards .`

ref `X-WE DESC-ALSO NEED TO LOOK DESC-FORWARDS .`

unc `X-WE ALSO NEED TO LOOK DESC-FORWARDS .`

con `X-WE ALSOGREENTRADE LOOK FORTHWITH .`

Garden path on an unreachable reference (BART and QLoRA). `RUBBER` is not in the training vocabulary, so the free output, although exactly right, is illegal, and both constrained decoders fail on it in their own way.

src `we complain that belarus has a weak rubber stamp parliament , but look around you .`

ref `X-WE COMPLAIN THAT BELARUS HAVE DESC-WEAK RUBBER STAMP PARLIAMENT , BUT LOOK AROUND X-YOU .`

unc `(both systems: correct)`

con `BART: ... DESC-WEAK RUB BINGO STAMP PARLIAMENT , ...`

con `QLoRA: ... DESC-WEAK RUB BER?S PARLIAMENT , ...`

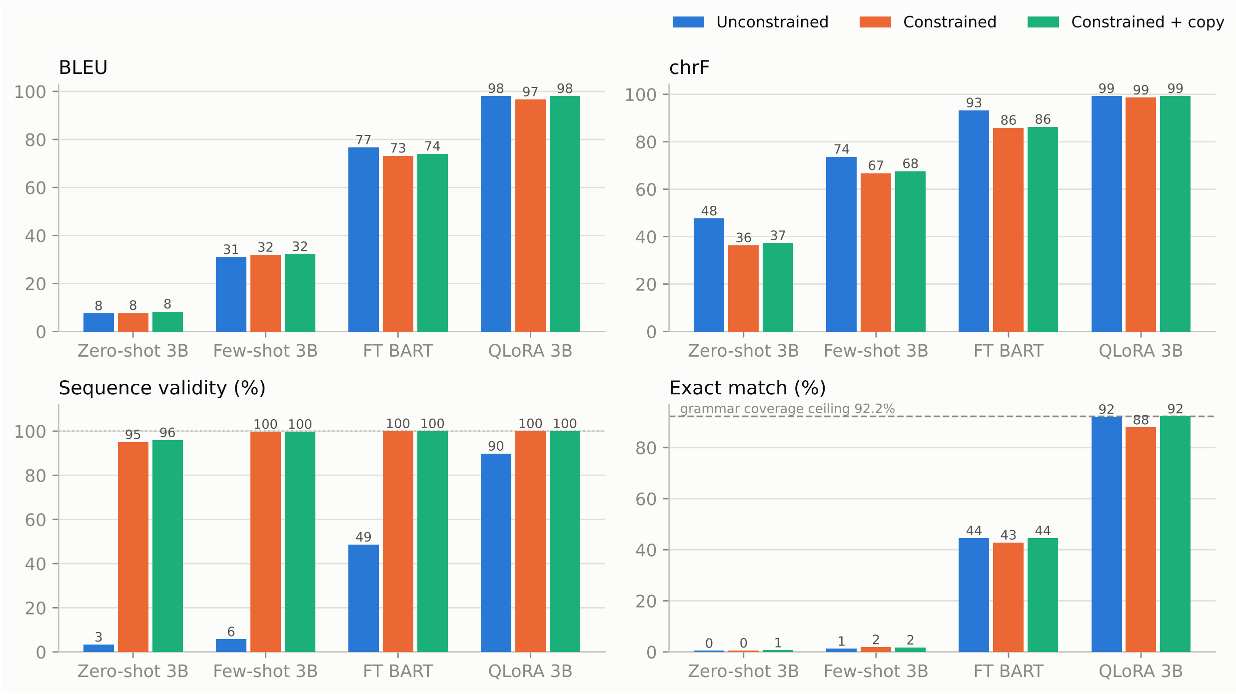

Figure 5: Quality and validity across systems and conditions (data of Table 3).

System	Cond.	BLEU	chrF	Exact	Seq. val.	Exact reach. (/922)	Exact unreach. (/78)
Fine-tuned BART	unc	76.68	93.26	44.5	48.6	426	19
	con v2	74.36	85.00	42.9	100.0	429	0
	con v3	73.24	85.90	42.8	100.0	428	0
	copy v2	75.03	85.32	44.3	100.0	429	14
	copy v3	74.05	86.29	44.5	100.0	428	17
QLoRA Llama 3B	unc	98.08	99.28	92.0	89.8	870	50
	con v2	96.68	98.49	88.0	100.0	880	0
	con v3	96.68	98.64	87.9	100.0	879	0
	copy v2	97.74	99.00	91.2	100.0	879	33
	copy v3	98.04	99.27	92.2	100.0	877	45

Table 4: Grammar-version ablation on the fine-tuned systems. The last two columns split the exact matches by whether grammar v2 can express the reference (922 reachable / 78 unreachable); each row is a separate decoding run under its own grammar (none, v2 or v3), so the wider grammars can score in the unreachable column. Validity uses the grammar of the row (v3 for *unc*).

		Hybrid		Always constrained	
System, fallback	Fallb.	BLEU	Exact	BLEU	Exact
BART, con	514	73.42	43.4	73.24	42.8
BART, copy	514	74.23	45.1	74.05	44.5
QLoRA, con	102	96.68	87.9	96.68	87.9
QLoRA, copy	102	98.04	92.2	98.04	92.2

Table 5: Hybrid decoding (unconstrained output if valid, else the constrained output; grammar v3) against always-constrained decoding. "Fallbacks" is the number of invalid unconstrained outputs.

A − B	ΔBLEU CI	ΔExact CI
Few-shot: unc − con	[-1.93, +0.16]	[-1.1, -0.1]
BART: unc − con	[+2.73, +4.18]	[+0.7, +2.9]
BART: con − copy	[-1.11, -0.53]	[-2.6, -1.0]
QLoRA: unc − con	[+1.00, +1.83]	[+2.7, +5.7]
QLoRA: unc − copy	[-0.24, +0.29]	[-1.1, +0.7]
QLoRA: copy v3 − copy v2	[+0.14, +0.49]	[+0.3, +1.8]

Table 6: Paired bootstrap 95% confidence intervals (1,000 resamples, seed 0) on differences between conditions. A positive value means the first condition is better. Grammar v3 unless stated.

Garden path that looks plausible (BART). `SIDESTEP` **is out of vocabulary; the forced** `SIDELINE` **reads well, and the derailment shows up two tokens later.**

src `you have tried to sidestep the issue and you have done a balancing act .`

ref `X-YOU HAVE TRY TO SIDESTEP ISSUE AND X-YOU HAVE DO BALANCE ACT .`

unc `(correct)`

con `X-YOU HAVE TRY TO SIDELINE ISSUE AND X-YOU SHOULD DO BALANCE ACT .`

Garden path on a proper noun (QLoRA). The same mechanism at saturation: the name is split into two legal words.

src `unfortunately , the marpol 73 / 78 convention was not enforced adequately either .`

ref `DESC-UNFORTUNATELY , MARPOL 73 78 CONVENTION BE DESC-NOT ENFORCE DESC-ADEQUATELY EIR .`

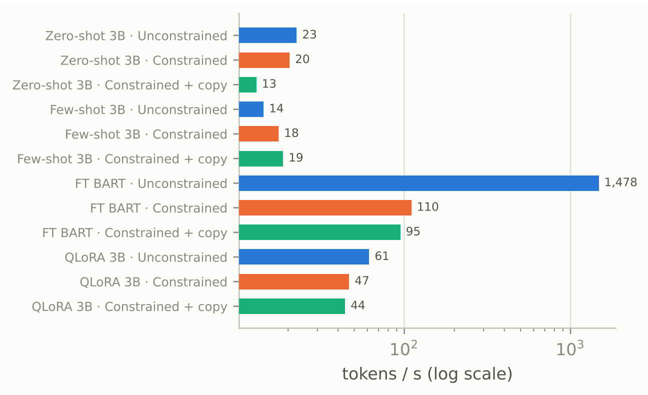

Figure 6: Decoding speed with and without the grammar mask.

unc `(correct)`

con `DESC-UNFORTUNATELY , MAR POLYCHLORINATE 73 78 CONVENTION BE DESC-NOT ENFORCE DESC-ADEQUATELY EIR .`

Constraint fixes a near-miss (QLoRA). The intended word is a legal prefix of the right one, so the mask completes it.

src `it is vital to allow unimpeded access for humanitarian assistance and aid to gaza without delay .`

ref `X-IT BE DESC-VITAL TO ALLOW DESC-UNIMPEDED ACCESS FOR ...`

unc `X-IT BE DESC-VITAL TO ALLOW DESC-UNIMPED ACCESS FOR ...`

con `X-IT BE DESC-VITAL TO ALLOW DESC-UNIMPEDED ACCESS FOR ...`

Constraint fixes a character-level garble (BART).

src `islamist jihadi extremists pose a threat to our very way of life .`

ref `ISLAMIST JIHADUS EXTREMIST POSE THREAT TO X-WE DESC-VERY WAY LIFE .`

unc `ISLAMIST JIJIRAQIST EXTREMIST ...`

con `ISLAMIST JIHADUS EXTREMIST ...`

Zero-shot loops; the constraint cannot fix a wrong plan.

src `we should behave outside european waters in just the same way as we do at home .`

con `WE BEHAVE DESC-SAME WAY X-OURSELVES AT X-HIMSELF X-HERSELF AT X-OURSELVES AT ... (until the token budget)`

5 Conclusions and future work

I compared four English to ASL-gloss systems, with and without grammar-constrained decoding, on a leakage-controlled split of ASLG-PC12. The findings are summarized below:

- Constrained decoding gives valid gloss output on every system, and even fine-tuned models need it: free decoding leaves the vocabulary in half of BART's outputs and in about one in ten of the QLoRA model's.
- The quality cost of the constraint shrinks as the model improves, from about 11 chrF for zero-shot prompting to under one point for the QLoRA model. On weak models the cost is garden path, and it shows up in character-level metrics rather than in BLEU.
- On the best model the remaining cost is entirely a coverage and data problem: the constraint is at least as accurate as free decoding on every reference the grammar can express, and loses only on the references it cannot express.
- Widening the grammar at decoding time raises the coverage ceiling from 92.2% to 97.7%, making the constrained QLoRA model statistically indistinguishable from the unconstrained one, with 100% valid output.
- Fine-tuning dominates prompting (7.6, 31, 77 and 98 BLEU), and few-shot demonstrations teach the output format that explicit instructions fail to convey.

Limitations. Every configuration was trained and decoded once; the bootstrap intervals are calculated on sampling variation over the 1,000 test sentences, so it does not capture training variance. The full test split was never decoded. All decoding is greedy.

Future work. Constrained beam search is the natural next step: it would let the decoder recover from a garden path by keeping the alternative legal prefix alive. The pattern should be tested on a human-annotated, non-templatic corpus (PHOENIX-2014T for German Sign Language [2], or an out-of-domain ASL test set). A fingerspelling rule would be the linguistically right treatment of unknown names.

Reproduction

All code is a small package (`src/glosstrans`: data loading, prompts, grammar construction and caching, the seq2seq logits processor, metrics) plus one script per stage in `scripts/`, at `https://github.com/TheRealGioviok/asl-gloss-nlp`. The whole pipeline is chained in `scripts/run_all.ps1`; every run writes a JSON file with its configuration, metrics, throughput, grammar version and all 1,000 (source, reference, prediction) triples, and every table of this report is generated from those files by `scripts/make_report_tables.py`.

Pretrained models. The two fine-tuned systems are published on the Hugging Face Hub, so the demo and the evaluation can be run without retraining: the fine-tuned BART at `https://huggingface.co/425GMM/bart-base-aslg-gloss` (full weights in fp16, 269 MB) and the QLoRA adapter at `https://huggingface.co/425GMM/llama-3.2-3b-qlora-aslg-gloss` (adapter only, 109 MB; the base model is fetched from its Hub mirror). The evaluation script and the interactive demo look for the local training output in `models/` and fall back to these repositories otherwise, so on a fresh clone the first run downloads them. Explicit download, into the paths the training scripts would have used:

```
hf download 425GMM/bart-base-aslg-gloss \
   --local-dir models/bart-base-aslg
hf download 425GMM/llama-3.2-3b-qlora-aslg-gloss \
   --local-dir models/llama3b-qlora-aslg
```

The interactive demo, `scripts/translate_repl.py -models both`, loads both systems (about 8 GB of VRAM) and decodes every typed English sentence under the three conditions of Table 3: unconstrained, constrained, and constrained with the copy rule. Each model card on hugging face also carries the training settings, the results, and a standalone loading snippet for use outside this code base.

References

[1] W. C. Stokoe. Sign language structure: An outline of the visual communication systems of the American deaf. *Studies in Linguistics, Occasional Papers 8*, University of Buffalo, 1960. Reprinted in *Journal of Deaf Studies and Deaf Education* 10(1), 2005. https://academic.oup.com/jdsde/article-abstract/10/1/3/361306

[2] N. C. Camgöz, S. Hadfield, O. Koller, H. Ney, and R. Bowden. Neural sign language translation. In *CVPR*, 2018. https://openaccess.thecvf.com/content_cvpr_2018/html/Camgoz_Neural_Sign_Language_CVPR_2018_paper.html

[3] M. Müller, Z. Jiang, A. Moryossef, A. Rios, and S. Ebling. Considerations for meaningful sign language machine translation based on glosses. In *ACL (Short Papers)*, pages 682–693, 2023. https://aclanthology.org/2023.acl-short.60/

[4] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *ACL*, 2020. https://aclanthology.org/2020.acl-main.703/

[5] A. Grattafiori et al. The Llama 3 herd of models. *arXiv:2407.21783*, 2024. https://arxiv.org/abs/2407.21783

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *NeurIPS*, 2023. https://arxiv.org/abs/2305.14314

[7] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: a method for automatic evaluation of machine translation. In *ACL*, 2002. https://aclanthology.org/P02-1040/

[8] M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In *WMT*, 2015. https://aclanthology.org/W15-3049/

[9] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out (ACL workshop)*, 2004. https://aclanthology.org/W04-1013/

[10] M. Post. A call for clarity in reporting BLEU scores. In *WMT*, 2018. https://aclanthology.org/W18-6319/

[11] B. T. Willard and R. Louf. Efficient guided generation for large language models. *arXiv:2307.09702*, 2023. https://arxiv.org/abs/2307.09702

[12] S. Geng, M. Josifoski, M. Peyrard, and R. West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *EMNLP*, pages 10932–10952, 2023. https://aclanthology.org/2023.emnlp-main.674/

[13] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao, and T. Chen. XGrammar: Flexible and efficient structured generation engine for large language models. *arXiv:2411.15100*, 2024. https://arxiv.org/abs/2411.15100

[14] A. Othman and M. Jemni. English-ASL gloss parallel corpus 2012: ASLG-PC12. In *Proceedings of the LREC 2012 5th Workshop on the Representation and Processing of Sign Languages*, Istanbul, 2012. https://www.sign-lang.uni-hamburg.de/lrec/pub/12019.pdf

[15] P. Koehn. Europarl: A parallel corpus for statistical machine translation. In *MT Summit X*, 2005. https://aclanthology.org/2005.mtsummit-papers.11/

[16] Kaggle dataset "Unlock the power of English–ASL with ASLG-PC12" (redistribution of ASLG-PC12). https://www.kaggle.com/datasets/thedevastator/unlock-the-power-of-english-asl-with-aslg-pc12-c